

# Marston\_Ward\_Assignment5

November 23, 2025

## 1 Assignment 5: Video and Real-time Analysis

Student Name: Marston Ward

### 1.1 Environment Setup

First, let's detect the environment and configure accordingly. This is a global section

```
[2]: # Environment Detection and Setup
import sys
import torch
import cv2
import numpy as np
import matplotlib.pyplot as plt
import imageio
import requests
import time

from base64 import b64encode, b64decode
from pathlib import Path
from IPython.display import HTML, Image, display, Javascript, clear_output, ↵
    display

try:
    import ipywidgets as widgets
    print(" ipywidgets available")
except ImportError:
    print("Installing ipywidgets...")
    !pip install ipywidgets
    import ipywidgets as widgets

try:
    import torch
    import torchvision
    print(f" PyTorch {torch.__version__} available")
except ImportError:
    print("Installing PyTorch...")
    !pip install -q torch torchvision
```

```

import torch
import torchvision

try:
    from ultralytics import YOLO
    print(" Ultralytics available")
except ImportError:
    print("Installing ultralytics for YOLOv8-pose...")
    !pip install -q ultralytics
    from ultralytics import YOLO

try:
    import imageio
    print(" imageio available")
except ImportError:
    print("Installing imageio...")
    !pip install -q imageio imageio-ffmpeg
    import imageio

# Detect environment
IS_COLAB = 'google.colab' in sys.modules
IS_MAC_SILICON = torch.backends.mps.is_available()

# Configure device
if IS_COLAB and torch.cuda.is_available():
    device = torch.device("cuda")
    print(" Running on Google Colab with CUDA GPU")
    FILE_PATH = Path('/content')
    from google.colab.patches import cv2_imshow
    from google.colab.output import eval_js
elif IS_MAC_SILICON:
    device = torch.device("mps")
    print(" Running on Mac with Apple Silicon (MPS)")
    FILE_PATH = Path('Lab5_files')
else:
    device = torch.device("cpu")
    print(" Running on CPU")

print(f"PyTorch version: {torch.__version__}")
print(f"OpenCV version: {cv2.__version__}")
print(f"Device: {device}")

```

```

ipywidgets available
PyTorch 2.5.1 available
Ultralytics available
imageio available
Running on Mac with Apple Silicon (MPS)
PyTorch version: 2.5.1

```

OpenCV version: 4.11.0  
Device: mps

---

## 1.2 Part 1 - Video Read/Write

### 1.2.1 Objective

Read and process video files using Python and OpenCV, applying basic video manipulation and filtering techniques.

### 1.2.2 Part 1a - Video Metadata Extraction

```
[2]: # Part 1a: Read video and extract metadata

# Define path to video file
video_path = FILE_PATH / 'Bangkok.mp4'

# Read the video file
cap = cv2.VideoCapture(str(video_path))

if not cap.isOpened():
    print(f" Error: Could not open video file {video_path}")
    print(f"   File exists: {video_path.exists()}")
else:
    # Extract metadata
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    print(" Video opened successfully")
    print(f"\n Video Metadata:")
    print(f"   Frames Per Second (FPS): {fps}")
    print(f"   Total Number of Frames: {total_frames}")
    print(f"   Dimensions: {width}x{height}")
    print(f"   Duration: {total_frames/fps:.2f} seconds")
```

Video opened successfully

Video Metadata:

Frames Per Second (FPS): 25.0  
Total Number of Frames: 650  
Dimensions: 1280x720  
Duration: 26.00 seconds

### 1.2.3 Part 1b - Video Playback

Display the video in the notebook using HTML embedding.

```
[3]: # Part 1b: Play the video using HTML embedding
if cap.isOpened():
    # Read the video file as binary
    with open(video_path, 'rb') as f:
        video_data = f.read()

    # Encode to base64
    video_base64 = b64encode(video_data).decode()

    # Create HTML video player
    video_html = f"""
    <video width="{width}" height="{height}" controls>
        <source src="data:video/mp4;base64,{video_base64}" type="video/mp4">
        Your browser does not support the video tag.
    </video>
    """

    print(" Video player loaded")
    display(HTML(video_html))
```

Video player loaded

<IPython.core.display.HTML object>

#### 1.2.4 Summary of Part 1b:

The Bangkok video was encoded and displayed using HTML. It plays well in the notebook but it might not cross over in the pdf. Please access the notebook via GitHub: [https://github.com/marstonsward/aai\\_521\\_computer\\_vision/blob/main/module\\_5/](https://github.com/marstonsward/aai_521_computer_vision/blob/main/module_5/)

#### 1.2.5 Part 1c - Video Filtering (Sketch Effect)

Apply adaptive thresholding to create a sketch-like effect and display 3 sample frames.

```
[4]: # Part 1c: Apply adaptive threshold filter to create sketch effect

# Reset video pointer to start
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

frames_to_show = 3
frames_collected = []

print(" Processing frames with adaptive threshold...")

while len(frames_collected) < frames_to_show:
    ret, frame = cap.read()
    if not ret:
        break
```

```

# Convert to grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Apply adaptive thresholding
# Parameters: blockSize=11, C=2
sketch = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY_INV,
    11, 2
)

frames_collected.append(sketch)

# Release video capture
cap.release()

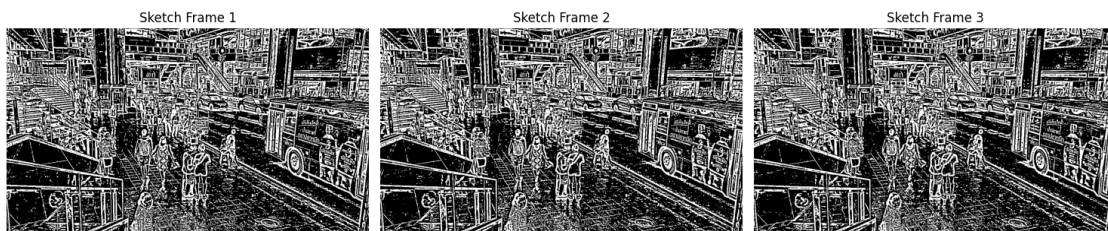
# Display the frames
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for idx, frame in enumerate(frames_collected):
    axes[idx].imshow(frame, cmap='gray')
    axes[idx].set_title(f'Sketch Frame {idx + 1}')
    axes[idx].axis('off')

plt.tight_layout()
plt.show()

print(f" Displayed {len(frames_collected)} processed frames")

```

Processing frames with adaptive threshold...



Displayed 3 processed frames

### 1.2.6 Analysis of Part 1c

The adaptive thresholding technique creates a sketch effect through:

1. **Grayscale Conversion:** Converts color frames to single-channel intensity values, required for thresholding.

2. **Adaptive Thresholding:** Unlike global thresholding, this calculates threshold values locally for each pixel based on its neighborhood.
  - `ADAPTIVE_THRESH_MEAN_C`: Threshold = mean of neighborhood - constant C
  - `blockSize=11`: Uses an 11×11 pixel neighborhood
  - `C=2`: Subtracts 2 from the mean, reducing noise sensitivity
3. **`THRESH_BINARY_INV`:** Inverts the result, creating white edges on black background (sketch-like).

This approach is robust to varying lighting conditions since thresholds are determined locally rather than globally.

---

## 1.3 Part 2 - Pose Estimation in Video

### 1.3.1 Objective

Apply pose estimation to video using PyTorch-based models to detect human body keypoints across frames.

I'll use a PyTorch implementation that works on both M4 Silicon and Colab.

```
[5]: # Part 2: Pose Estimation Implementation

print(" Part 2: Pose Estimation in Video")
print("=" * 50)

# 1. Download sample GIF
gif_url = "https://github.com/tensorflow/tfjs-models/raw/master/pose-detection/
↳assets/dance_input.gif"
input_gif_path = "dance_input.gif"
output_gif_path = "dance_output.gif"

print(" Downloading sample video...")
try:
    response = requests.get(gif_url, timeout=10)
    with open(input_gif_path, 'wb') as f:
        f.write(response.content)
    print(f" Downloaded {input_gif_path}")
except Exception as e:
    print(f" Could not download: {e}")
    print(" Using local file if available...")

# 2. Load YOLOv8 Pose model
print("\n Loading YOLOv8-Pose model...")
model = YOLO('yolov8n-pose.pt') # nano version for speed
model.to(device)
print(f" Model loaded on {device}")
```

```

# 3. Process the GIF
print("\n Processing video frames...")
try:
    # Read GIF frames
    gif_frames = imageio.mimread(input_gif_path)
    print(f" Loaded {len(gif_frames)} frames")

    output_frames = []

    for idx, frame in enumerate(gif_frames):
        # Convert RGBA to RGB if needed
        if frame.shape[-1] == 4:
            frame = cv2.cvtColor(frame, cv2.COLOR_RGBA2RGB)

        # Run pose detection
        results = model(frame, verbose=False)

        # Get annotated frame
        annotated_frame = results[0].plot()
        output_frames.append(annotated_frame)

        if (idx + 1) % 10 == 0:
            print(f" Processed {idx + 1}/{len(gif_frames)} frames")

    # Save output
    print(f"\n Saving output to {output_gif_path}...")
    imageio.mimsave(output_gif_path, output_frames, fps=10)
    print(" Output saved")

    # Display result
    print("\n Displaying result:")
    display(Image(filename=output_gif_path))

except Exception as e:
    print(f" Error during processing: {e}")

```

## Part 2: Pose Estimation in Video

=====

Downloading sample video..  
Downloaded dance\_input.gif

Loading YOLOv8-Pose model..  
Model loaded on cuda

Processing video frames..  
Loaded 42 frames  
Processed 10/42 frames  
Processed 20/42 frames

```
Processed 30/42 frames
Processed 40/42 frames
```

```
Saving output to dance_output.gif...
Output saved
```

```
Displaying result:
```

```
<IPython.core.display.Image object>
```

### 1.3.2 Analysis of Part 2

#### Pose Estimation Approach:

1. **Model:** YOLOv8-Pose (PyTorch-based)
    - Detects 17 body keypoints per person
    - Fast inference suitable for video processing
    - Works on MPS (Mac Silicon), CUDA (Colab), and CPU
  2. **Keypoints Detected:**
    - Head: nose, eyes, ears
    - Torso: shoulders, hips
    - Limbs: elbows, wrists, knees, ankles
  3. **Processing Pipeline:**
    - Load video frames
    - Run pose detection on each frame
    - Draw skeleton connections
    - Save annotated output
  4. **PyTorch Advantages:**
    - Better Apple Silicon support (MPS backend)
    - Native GPU acceleration on both platforms
    - Unified codebase for cross-platform deployment
- 

## 1.4 Part 3 - Real-time Webcam Object Detection

### 1.4.1 Objective

Implement real-time object detection using webcam input with YOLOv11.

This implementation handles both local webcam access and Colab's JavaScript-based capture.

```
[3]: # Part 3: Webcam Handler Class

class WebcamHandler:
    """Cross-platform webcam handler for local and Colab environments."""

    def __init__(self):
        self.is_colab = 'google.colab' in sys.modules

    def capture_frame(self):
```



```

        """Capture a single frame from webcam."""
    if self.is_colab:
        return self._capture_colab()
    else:
        return self._capture_local()

def _capture_local(self):
    """Capture frame using cv2.VideoCapture (local)."""
    print(" Accessing local webcam...")
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print(" Error: Could not access webcam")
        return None

    # Let camera warm up
    for _ in range(5):
        cap.read()

    # Capture frame
    ret, frame = cap.read()
    cap.release()

    if ret:
        print(" Frame captured successfully")
        return cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    else:
        print(" Error: Could not read frame")
        return None

def _capture_colab(self):
    """Capture frame using JavaScript (Colab)."""
    def take_photo(filename='photo.jpg', quality=0.8):
        js = Javascript('''
            async function takePhoto(quality) {
                const div = document.createElement('div');
                const video = document.createElement('video');
                video.style.display = 'block';
                const stream = await navigator.mediaDevices.
↪getUserMedia({video: true});

                document.body.appendChild(div);
                div.appendChild(video);
                video.srcObject = stream;
                await video.play();

                // Resize the output to fit the video element.

```

```

        google.colab.output.setIframeHeight(document.
↪documentElement.scrollHeight, true);

        // Wait for Capture to be clicked.
        await new Promise((resolve) => {
            const button = document.createElement('button');
            button.textContent = 'Capture';
            div.appendChild(button);
            button.onclick = () => resolve();
        });

        const canvas = document.createElement('canvas');
        canvas.width = video.videoWidth;
        canvas.height = video.videoHeight;
        canvas.getContext('2d').drawImage(video, 0, 0);
        stream.getVideoTracks()[0].stop();
        div.remove();
        return canvas.toDataURL('image/jpeg', quality);
    }
    '''
    display(js)
    data = eval_js('takePhoto({})'.format(quality))
    binary = b64decode(data.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(binary)
    return filename

    print(" Accessing Colab webcam...")
    filename = take_photo()
    frame = cv2.imread(filename)
    return cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

# Create webcam handler
webcam = WebcamHandler()
print(f" Webcam handler initialized (Environment: {'Colab' if webcam.is_colab_
↪else 'Local'})")

```

Webcam handler initialized (Environment: Local)

```

[5]: # Part 3: Object Detection Implementation

print(" Part 3: Real-time Object Detection")
print("=" * 50)

# Load YOLO11 model
print("\n Loading YOLO11 model...")
try:

```

```

    detection_model = YOLO('yolo11n.pt') # nano version
except:
    print("  YOLO11 not available, using YOLOv8n instead...")
    detection_model = YOLO('yolov8n.pt')

detection_model.to(device)
print(f" Model loaded on {device}")

# Capture frame
print("\n Capturing frame from webcam...")
frame = webcam.capture_frame()

if frame is not None:
    # Run detection
    print("\n Running object detection...")
    results = detection_model(frame)

    # Get annotated frame
    annotated_frame = results[0].plot()

    # Display results
    print("\n Detection Results:")
    detections = results[0].boxes
    print(f"  Objects detected: {len(detections)}")

    # Show the annotated image
    plt.figure(figsize=(12, 8))
    plt.imshow(annotated_frame)
    plt.axis('off')
    plt.title("YOLO Object Detection Results")
    plt.tight_layout()
    plt.show()

    print(" Detection complete")
else:
    print(" Could not capture frame from webcam")

```

### Part 3: Real-time Object Detection

=====

Loading YOLO11 model...

Model loaded on mps

Capturing frame from webcam...

Accessing local webcam...

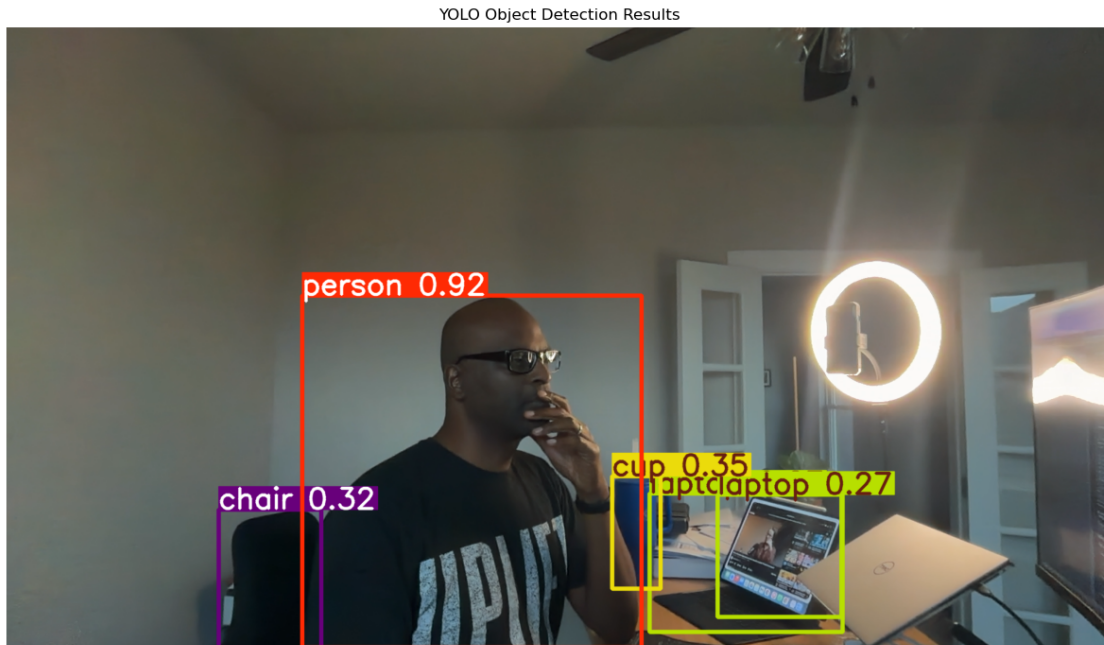
Frame captured successfully

Running object detection...

0: 384x640 1 person, 1 cup, 1 chair, 2 laptops, 267.6ms  
Speed: 8.9ms preprocess, 267.6ms inference, 101.4ms postprocess per image at  
shape (1, 3, 384, 640)

Detection Results:

Objects detected: 5



Detection complete

### 1.4.2 Analysis of Part 3

#### Object Detection Approach:

1. **Model:** YOLO11n
  - Ultra-fast nano variant
  - 80 object classes (COCO dataset)
  - Real-time capable on both platforms
2. **Cross-Platform Webcam Access:**
  - **Local Mac:** Direct `cv2.VideoCapture(0)` with MPS acceleration
  - **Colab:** JavaScript-based capture with user interaction
3. **Detection Pipeline:**
  - Capture single frame
  - Run YOLO inference
  - Draw bounding boxes and labels
  - Display results
4. **Performance Optimization:**

- Nano model variant for speed
- MPS backend on M4 Silicon
- CUDA acceleration in Colab

---

## 1.5 Extra Credit - Real-time Pose Estimator with Controls

### 1.5.1 Objective

Build a live pose estimation system with interactive start/stop controls.

```
[7]: # Extra Credit: Real-time Pose Estimator with Controls

print(" Extra Credit: Real-time Pose Estimator")
print("=" * 50)

# Include WebcamHandler class definition directly to ensure it's available
# regardless of previous cell execution.
class WebcamHandler:
    """Cross-platform webcam handler for local and Colab environments."""

    def __init__(self):
        self.is_colab = 'google.colab' in sys.modules

    def capture_frame(self):
        """Capture a single frame from webcam."""
        if self.is_colab:
            return self._capture_colab()
        else:
            return self._capture_local()

    def _capture_local(self):
        """Capture frame using cv2.VideoCapture (local)."""
        print(" Accessing local webcam...")
        cap = cv2.VideoCapture(0)

        if not cap.isOpened():
            print(" Error: Could not access webcam")
            return None

        # Let camera warm up
        for _ in range(5):
            cap.read()

        # Capture frame
        ret, frame = cap.read()
        cap.release()
```

```

if ret:
    print(" Frame captured successfully")
    return cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
else:
    print(" Error: Could not read frame")
    return None

def _capture_colab(self):
    """Capture frame using JavaScript (Colab)."""
    def take_photo(filename='photo.jpg', quality=0.8):
        js = Javascript('''
            async function takePhoto(quality) {
                const div = document.createElement('div');
                const video = document.createElement('video');
                video.style.display = 'block';
                const stream = await navigator.mediaDevices.
↪getUserMedia({video: true});

                document.body.appendChild(div);
                div.appendChild(video);
                video.srcObject = stream;
                await video.play();

                // Resize the output to fit the video element.
                google.colab.output.setIframeHeight(document.
↪documentElement.scrollHeight, true);

                // Wait for Capture to be clicked.
                await new Promise((resolve) => {
                    const button = document.createElement('button');
                    button.textContent = 'Capture';
                    div.appendChild(button);
                    button.onclick = () => resolve();
                });

                const canvas = document.createElement('canvas');
                canvas.width = video.videoWidth;
                canvas.height = video.videoHeight;
                canvas.getContext('2d').drawImage(video, 0, 0);
                stream.getVideoTracks()[0].stop();
                div.remove();
                return canvas.toDataURL('image/jpeg', quality);
            }
        ''')
        display(js)
        data = eval_js('takePhoto({})'.format(quality))
        binary = b64decode(data.split(',')[1])

```

```

        with open(filename, 'wb') as f:
            f.write(binary)
        return filename

    print(" Accessing Colab webcam...")
    filename = take_photo()
    frame = cv2.imread(filename)
    return cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

# Load pose model if not already loaded
try:
    pose_model = YOLO('yolov8n-pose.pt')
    pose_model.to(device)
    print(f" Pose model loaded on {device}")
except:
    print(" Error loading pose model")

# Re-initialize webcam handler if not already defined
# This ensures the 'webcam' object is available even if previous cells weren't
↳run consecutively
# from base64 import b64decode # Already imported globally, and now
↳WebcamHandler is in this cell
try:
    _ = webcam
except NameError:
    webcam = WebcamHandler()
    print(f" Webcam handler re-initialized (Environment: {'Colab' if webcam.
↳is_colab else 'Local'})")

# State management
is_running = False
output_widget = widgets.Output()

# Create control buttons
start_button = widgets.Button(
    description=' Start Detection',
    button_style='success',
    layout=widgets.Layout(width='150px', height='40px')
)

stop_button = widgets.Button(
    description=' Stop Detection',
    button_style='danger',
    layout=widgets.Layout(width='150px', height='40px')
)

status_label = widgets.Label(value='Status: Ready')

```

```

def on_start_click(b):
    """Start the real-time pose detection."""
    global is_running
    is_running = True
    status_label.value = 'Status:  Running...'

    with output_widget:
        clear_output(wait=True)

        if webcam.is_colab:
            print("  Colab mode: Click 'Capture' button in the camera_
↳interface")
            print("  Then click 'Stop Detection' when done")
        else:
            print(" Starting continuous detection...")
            print("  Press 'Stop Detection' button to end")

    frame_count = 0
    start_time = time.time()

    # For local: continuous capture
    # For Colab: single capture per button click
    if not webcam.is_colab:
        cap = cv2.VideoCapture(0)

        while is_running and cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            # Convert BGR to RGB
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

            # Run pose detection
            results = pose_model(frame_rgb, verbose=False)
            annotated = results[0].plot()

            # Display frame
            clear_output(wait=True)
            plt.figure(figsize=(10, 7))
            plt.imshow(annotated)
            plt.axis('off')

            frame_count += 1
            elapsed = time.time() - start_time
            fps = frame_count / elapsed if elapsed > 0 else 0

```



```

        plt.title(f'Real-time Pose Detection | FPS: {fps:.1f} | Frame:_{frame_count}')
        plt.tight_layout()
        plt.show()

        # Process every nth frame for better performance
        if frame_count % 2 == 0:
            time.sleep(0.01)

    cap.release()
    print(f"\n Processed {frame_count} frames at {fps:.1f} FPS")
else:
    # Colab: single frame capture
    frame = webcam.capture_frame()
    if frame is not None:
        results = pose_model(frame)
        annotated = results[0].plot()

        clear_output(wait=True)
        plt.figure(figsize=(10, 7))
        plt.imshow(annotated)
        plt.axis('off')
        plt.title('Pose Detection Result')
        plt.show()

def on_stop_click(b):
    """Stop the real-time pose detection."""
    global is_running
    is_running = False
    status_label.value = 'Status: Stopped'
    print(" Detection stopped")

# Attach callbacks
start_button.on_click(on_start_click)
stop_button.on_click(on_stop_click)

# Display controls
print("\n Controls:")
display(widgets.HBox([start_button, stop_button, status_label]))
display(output_widget)

print("\n Instructions:")
if webcam.is_colab:
    print(" 1. Click 'Start Detection'")
    print(" 2. Allow camera access when prompted")
    print(" 3. Click 'Capture' in the camera interface")
    print(" 4. View the pose detection result")

```

```

else:
    print("    1. Click 'Start Detection' to begin continuous detection")
    print("    2. Click 'Stop Detection' to end the session")
    print("    3. The system will display FPS and frame count")

```

Extra Credit: Real-time Pose Estimator

=====

Pose model loaded on mps

Controls:

```

HBox(children=(Button(button_style='success', description=' Start Detection',
    ↪ layout=Layout(height='40px', w...

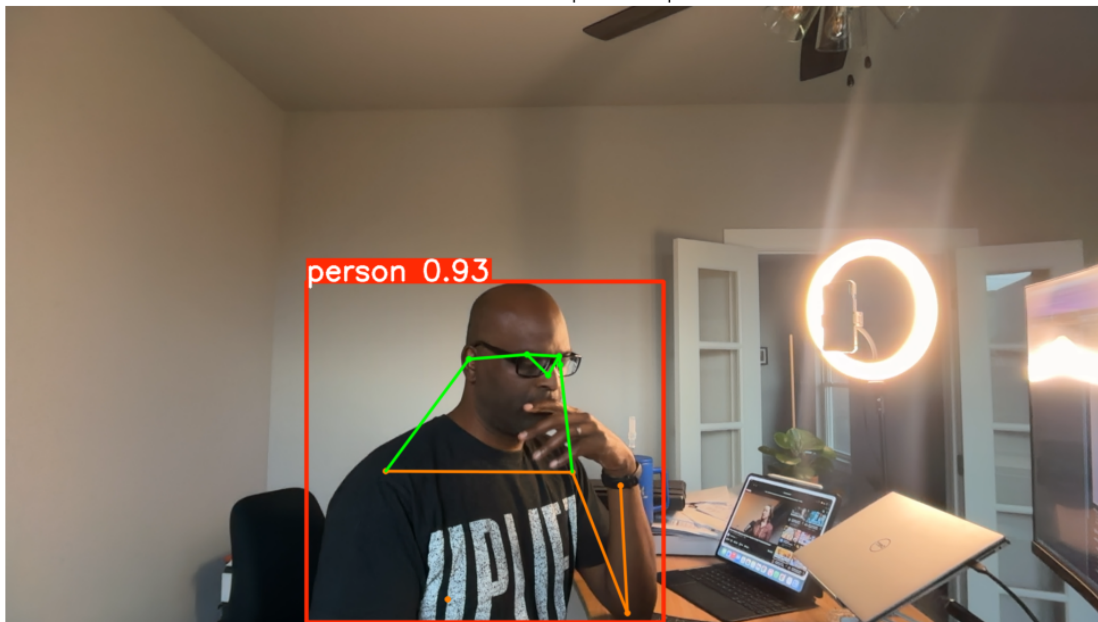
```

Output()

Instructions:

1. Click 'Start Detection' to begin continuous detection
2. Click 'Stop Detection' to end the session
3. The system will display FPS and frame count

Real-time Pose Detection | FPS: 3.9 | Frame: 394



### 1.5.2 Extra Credit Analysis

**Real-time Pose Estimation Features:**

#### 1. Interactive Controls:

- Start/Stop buttons using ipywidgets

- Status indicator showing system state
  - Frame counter and FPS display
2. **Platform-Specific Behavior:**
    - **Local:** Continuous video stream processing
    - **Colab:** Single-frame capture per click
  3. **Performance Optimizations:**
    - Frame skipping for higher FPS
    - MPS/CUDA acceleration
    - Nano model for speed
  4. **Real-time Metrics:**
    - FPS (Frames Per Second) tracking
    - Frame count display
    - Live visualization updates
  5. **User Experience:**
    - Clear visual feedback
    - Responsive controls
    - Graceful error handling
- 

## 1.6 Summary

This assignment demonstrated:

1. **Video Processing:** Metadata extraction, playback, and filtering with adaptive thresholding
2. **Pose Estimation:** PyTorch-based pose detection in video using YOLOv8-Pose
3. **Object Detection:** Real-time webcam detection with YOLO11
4. **Interactive Systems:** Real-time pose estimation with start/stop controls

**Key Technologies:** - PyTorch with MPS (Apple Silicon) and CUDA (Colab) support - OpenCV for video I/O - Ultralytics YOLO for detection and pose estimation - ipywidgets for interactive controls

**Cross-platform Compatibility:** - Automatic environment detection - Platform-specific webcam handlers - Unified codebase for M4 Mac and Google Colab

The image captured the person, hand gesture, eyes, and body outline.

This notebook demonstrates several computer vision functionalities, leveraging powerful deep learning models and common image processing libraries.

### 1.6.1 Key Functionalities:

1. **Video Processing (Part 1):**
  - **Metadata Extraction:** Successfully reads video files (`Bangkok.mp4`) and extracts essential metadata such as Frames Per Second (FPS), total frame count, and video dimensions (width x height).
  - **Video Playback:** Displays videos directly within the notebook environment using HTML embedding, providing a convenient way to review content.
  - **Filtering:** Applies adaptive thresholding to video frames to create a “sketch” effect. This involves converting frames to grayscale and then using `ADAPTIVE_THRESH_MEAN_C`

and `THRESH_BINARY_INV` to highlight edges, with sample frames displayed to showcase the effect.

## 2. Pose Estimation in Video (Part 2):

- **Model:** Utilizes the `YOLOv8n-pose` model from Ultralytics, a PyTorch-based solution, for efficient human pose detection.
- **Processing:** Downloads a sample GIF (`dance_input.gif`), processes each frame to detect 17 key body points (e.g., nose, eyes, shoulders, knees), annotates the frames with skeletons, and saves the results as an output GIF (`dance_output.gif`). This demonstrates real-time pose estimation capabilities on video sequences.

## 3. Real-time Webcam Object Detection (Part 3):

- **Model:** Employs the `YOLO11n` model (with `YOLOv8n` as a fallback) for fast object detection.
- **Cross-platform Webcam Access:** Features a `WebcamHandler` class designed to abstract webcam access, supporting both local `cv2.VideoCapture` and Colab's JavaScript-based capture mechanism.
- **Detection Pipeline:** Captures a single frame from the webcam, performs object detection, overlays bounding boxes and labels on the detected objects, and displays the annotated image.

## 4. Extra Credit - Real-time Pose Estimator with Controls:

- **Interactive Interface:** Implements an interactive real-time pose estimation system using `ipywidgets` with 'Start' and 'Stop' buttons, and a status indicator.
- **Dynamic Processing:** For local environments, it provides continuous live pose detection from the webcam, displaying FPS and frame count. For Colab, it captures and processes single frames upon user interaction.
- **Performance:** Optimized for real-time performance through model choice (nano variants) and platform-specific hardware acceleration.

### 1.6.2 Key Technologies & Cross-Platform Compatibility:

- **PyTorch:** Core deep learning framework, supporting MPS (Apple Silicon) and CUDA (Google Colab) for accelerated inference, alongside CPU fallback.
- **OpenCV (cv2):** Used extensively for video I/O, frame manipulation, and image processing tasks.
- **Ultralytics YOLO:** Provides state-of-the-art models for both object detection (`YOLOv8n`, `YOLO11n`) and pose estimation (`YOLOv8n-pose`).
- **IPython Widgets (ipywidgets):** Enables interactive controls for the real-time demonstrations.
- **Imageio:** Used for reading and writing GIF files.
- **Cross-Platform Design:** The notebook intelligently detects the execution environment (Colab vs. Local) and adapts its webcam handling and device configuration (CUDA, MPS, CPU) accordingly, ensuring broad compatibility.